

```

        }
        return count;
    }
    pos = idx + sub.length();
    count++;
    while ((idx = s.indexOf(sub, pos)) != -1) {
        int idx = 0;
        int pos = 0;
        int count = 0;
        return 0;
    }
    if (s == null || sub == null || "".equals(sub)) {
        private static int countOccurrencesOf(String s, String sub) {
            return false;
        }
        return false;
    }
    return true;
    if (!Character.isWhitespace(str.charAt(i)))
        for (int i = 0; i < strLen; i++) {
            return false;
        }
    if (str == null || (strLen = str.length()) == 0)
        int strLen = 0;
    public static boolean hasText(String str) {
        return new String(buf);
    }
}

    }
    buf[i] = padChar;
    for (int i = 0; i < bufLength; i++) {
        int bufLength = buf.length;
        if (null != buf) {
            char[] buf = new char[repeat];
            throw new IndexOutOfBoundsException("Cannot pad a negative amount: " + repeat);
            if (repeat < 0)
                private static String padding(int repeat, char padChar) throws
                    IndexOutOfBoundsException {
                        return buf.toString();
                    }
                }
                return buf.toString();
            }
            buf.append(mask);
            for (int i = 0; i < repeat; i++) {
                StringBuffer buf = new StringBuffer(outputLength);
                break;
            }
            default:
                return new String(output2);
            }
            k--;
            output2[k + 1] = ch1;
            output2[k] = ch0;
            for (k = repeat * 2 - 2; k >= 0; k--) {
                char output2[] = new char[outputLength];
                char ch1 = mask.charAt(1);
                char ch0 = mask.charAt(0);
                case 2:
                    return new String(output1);
                }
                output1[j] = ch;
            }
            for (j = repeat - 1; j >= 0; j--) {

```

```

    if (insting == null)
        return null;
    if (oldPattern == null || newPattern == null)
        return insting;
    StringBuffer sbuf = new StringBuffer();
    int pos = 0;
    int index = insting.indexOf(oldPattern);
    int patlen = oldPattern.length();
    while (index >= 0) {
        sbuf.append(insting.substring(pos, index));
        sbuf.append(newPattern);
        pos = index + patlen;
        index = insting.indexOf(oldPattern, pos);
    }
    sbuf.append(insting.substring(pos));
    return sbuf.toString();
}

public static String replace(String insting, String oldPattern, String newPattern) {
    return replace(insting, pattern, "");
}

public static String delete(String insting, String pattern) {
    return replace(insting, pattern, "");
}

private static String[] delimitedListToArray(String s, String delim) {
    if (s == null)
        return new String[0];
    if (delim == null)
        return new String[] {s};
    List<String> l = new LinkedList<>();
    int pos = 0;
    int delPos = 0;
    while ((delPos = s.indexOf(delim, pos)) != -1) {
        l.add(s.substring(pos, delPos));
        pos = delPos + delim.length();
    }
    if (pos <= s.length())
        l.add(s.substring(pos));
    return l.toArray(new String[l.size()]);
}

private static String arrayToDelimitedString(Object[] arr, String delim) {
    if (arr == null)
        return "null";
    StringBuffer sb = new StringBuffer();
    int arrLength = arr.length;
    for (int i = 0; i < arrLength; i++) {
        if (i > 0)
            sb.append(delim);
        sb.append(arr[i]);
    }
    return sb.toString();
}

private static String collectionToDelimitedString(Collection<?> c, String delim) {
    if (c == null)
        return "null";
    StringBuffer sb = new StringBuffer();
    Iterator<?> it = c.iterator();
    int i = 0;
    while (it.hasNext()) {
        if (i++ > 0)
            sb.append(delim);
        sb.append(it.next());
    }
    return sb.toString();
}

```

```

        }
        return string.matches(pattern);
    }

    static boolean isFormattedString(String string, String pattern) {
        if (string == null || pattern == null)
            return false;
        return string.matches(pattern);
    }

    private static boolean isDigit(String digit) {
        if (digit == null)
            return false;
        char[] chars = digit.toCharArray();
        int charsLength = chars.length;
        for (int i = 0; i < charsLength; i++)
            if (!Character.isDigit(chars[i]))
                return false;
        return true;
    }

    private static boolean isShangui(String inputStr, boolean full) {
        char[] chars = inputStr.toCharArray();
        int charsLength = chars.length;
        if (ifFull) {
            for (int j = 0; j < charsLength; j++)
                if (!isShangui(chars[j]))
                    return true;
        }
        return false;
    }

    private static boolean isShangui(String inputStr, boolean full) {
        char[] chars = inputStr.toCharArray();
        int charsLength = chars.length;
        for (int i = 0; i < charsLength; i++)
            if (!isShangui(chars[i]))
                return false;
        return true;
    }

    private static String changeFirstCharacterCase(boolean capitalize, String str) {
        int strLen = 0;
        if (str == null || (strLen = str.length()) == 0)
            return str;
        StringBuffer buf = new StringBuffer(strLen);
        if (capitalize) {
            buf.append(Character.toUpperCase(str.charAt(0)));
        } else {
            buf.append(Character.toLowerCase(str.charAt(0)));
        }
        buf.append(str.substring(1));
        return buf.toString();
    }

    public static String capitalize(String str) {
        return changeFirstCharacterCase(false, str);
    }

    public static String capitalize(String str) {
        return changeFirstCharacterCase(true, str);
    }

    }
    }
    return sb.toString();
}
sb.append(it.next());
sb.append(delim);

```